

CSCI 650 - Algorithms and Computability

Assignment 3

Solutions to the written questions on this assignment should be submitted via PDF to Canvas before **October 4th at 11:59 pm**. Your Turing machine simulator and your implementation of the CYK algorithm should be submitted to INGINious by the same deadline. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions **independently**. Feel free to communicate via [Discord](#) and to post questions on the appropriate forum in Canvas. Do not post solutions.

Have fun!

Problems

1. In this question we will write a simple function to simulate a Turing machine and use it to investigate $BB(2)$, the second busy beaver number.
 - (a) (10 pts) Write a function `simulateTM(delta, A, w)` which takes a transition function, a set of accepting states, and a string as input. This function should return a string associated with the final tape for the Turing machine along with a boolean indicating whether or not w is accepted. We will assume that A is the start state and that b is the blank symbol. Submit your code on INGINious and include a code snippet of this function in your PDF submission. `turingMachineTests.py`, a script meant to help with local testing, is available on Canvas.
 - (b) (15 pts) Consider the [busy beaver problem](#) as discussed in class. Write code to consider all Turing machines with two states, a binary input alphabet, and a tape initially filled with 0s. Augment your code from part (a) to count the number of steps each machine takes before halting up to a maximum of 100 steps. Create a table similar to the one below indicating the number and fraction of machines that take i steps to finish. Include a snippet of the code you used to find these values.

Steps	0	1	2	3	4	5	6	≥ 100
Number	3	8	4	12	42	112	57	239
Fraction	0.1	0.04	0.2	0.07	0.06	0.03	0.05	0.45
 - (c) ($BB(n)$ pts for $n \geq 6$) Extra credit - Find a Turing machine associated with $BB(n)$ and prove that it is correct.
2. (10 pts) Context-free grammars play an important role in many areas of computer science including in programming languages and compilers, natural language processing, and parsing among others.

Write a function `cyk(G, w)` that takes a context-free grammar and a string as input and returns a boolean indicating whether or not the string is in the language associated with the context-free grammar along with a table filled in according to the CYK algorithm as discussed in class. Submit your code on INGINious and include a code snippet of this function in your PDF submission. `cykTests.py`, a script meant to help with local testing, is available on Canvas.

3. Recall the definitions of the vertex cover and independent set problems.

Definition (Vertex Cover). *Given an undirected graph $G = (V, E)$, a **vertex cover** of G is a subset of vertices $C \subseteq V$ such that, for every edge $\{u, v\} \in E$, either $u \in C$ or $v \in C$.*

The Vertex Cover Problem

Input: An undirected graph $G = (V, E)$ and an integer k .

Output: True if and only if G has a vertex cover of size k or less.

Definition (Independent Set). *Given an undirected graph $G = (V, E)$, an **independent set** is a subset of vertices $I \subseteq V$ such that no two elements of I are adjacent. In particular, if $u, v \in I$, then $\{u, v\} \notin E$.*

The Independent Set Problem

Input: An undirected graph $G = (V, E)$ and an integer k .

Output: True if and only if G contains an independent set of size k or more.

- (a) (10 pts) Write pseudocode (using Python syntax) for a function `checkVertexCover(G, k, C)` that returns **True** if C is a vertex cover of G of size k or less and **False** otherwise. Analyze the asymptotic runtime of this function.
- (b) (10 pts) Write pseudocode (using Python syntax) for a function `findIndependentSet(G)` that finds an independent set of maximum size in G . You may assume access to a function `findVertexCover(G)` that is guaranteed to return a vertex cover of G of minimum size in polynomial time. In particular, `findIndependentSet(G)` must convert an arbitrary independent set instance to an instance of vertex cover. Analyze the asymptotic runtime of this function.

4. The set cover problem is defined as follows.

Definition (Set Cover). *Given a set of elements U (often called the universe) and subsets $S_1, S_2, \dots, S_m \subseteq U$, $C \subseteq \{1, 2, \dots, m\}$ is a **set cover** of U if $\bigcup_{i \in C} S_i = U$.*

The Set Cover Problem

Input: A universe U , sets $S_1, S_2, \dots, S_m \subseteq U$, and an integer k .

Output: True if and only if there is a cover C of U such that $|C| \leq k$.

- (a) (10 pts) Write pseudocode (using Python syntax) for a function `checkSetCover(U, S, k, C)` that returns **True** if C is a set cover of U of size k or less and **False** otherwise. Analyze the asymptotic runtime of this function.
- (b) (10 pts) Write pseudocode (using Python syntax) for a function `findVertexCover(G)` that finds a vertex cover of minimum size in G . You may assume access to a function `findSetCover(U, S)` that is guaranteed to return a minimum set cover of U in polynomial time. In particular, `findVertexCover(G)` must convert an arbitrary vertex cover instance to an instance of set cover. Analyze the asymptotic runtime of this function.